

MULTI-AGENT WORLD: SCALING MULTI-AGENT ORCHESTRATION TRAINING VIA RECURSIVE ENVIRONMENT IMPROVEMENT AND SCHEDULE-AWARE REWARDS

Anonymous authors

Paper under double-blind review

ABSTRACT

Multi-agent systems assign complementary subtasks to different agents, and their effectiveness depends on an orchestrator that partitions a request into subtasks, schedules them against their dependencies, and integrates the evidence that workers return. Training such an orchestrator requires multi-agent environments, in which a task provides subtasks that can be distributed across agents and a verifier that reports how they were distributed. Existing agentic environments are synthesized for a single agent, and they yield mostly linear call chains under terminal-only verification. An orchestrator trained inside them therefore has no scheduling decision to learn and no feedback on the decomposition it chose. We present *Multi-Agent World* (MAW), an end-to-end framework that develops multi-agent orchestration by coupling multi-agent environment synthesis with multi-agent reinforcement learning. MAW introduces *recursive environment improvement*, which recursively composes reference task graphs through serial and parallel operators inside single-agent environments, and turns each composed graph into one coherent request through semantic reconstruction and execution-grounded admission. The same composition record identifies which subtasks are verified as independent, which is what a *schedule-aware process reward* needs in order to score the recorded execution of the whole team. Trained under this reward with frozen workers, the orchestrator learns to decompose a request, run independent work concurrently, and integrate what the workers return. Experiments on six agentic benchmarks show that MAW improves task success by ___ over its single-agent initialization and shortens the execution critical path by ___.

1 INTRODUCTION

Large language model (LLM) agents now carry out workflows that combine information gathering, tool use, and multi-step reasoning (Yao et al., 2023; Schick et al., 2023; Qin et al., 2024). Such a workflow usually contains complementary subtasks whose outputs must be assembled into one solution, and multi-agent systems assign each subtask to a separate agent, as exemplified by AutoGen (Wu et al., 2023) and MetaGPT (Hong et al., 2024). Recent systems further execute independent subtasks concurrently, which shortens the critical path of a long workflow (Hadfield et al., 2025; Kimi Team, 2026). However, distributing a task across agents does not by itself produce a better solution. The benefit instead depends on three decisions, namely how the request is divided, when each part is executed, and how the returned results are combined. In a multi-agent system, one component makes all three, the orchestrator.

Building an effective multi-agent system is difficult, as these three decisions constrain one another. An ideal orchestrator first partitions a request into subtasks that one worker can finish alone. Subtasks that are too fine raise the coordination overhead, while subtasks that are too coarse reduce the team to a single agent. The orchestrator then separates the subtasks that consume the output of another subtask from the subtasks that are genuinely independent. For instance, a request that compares several vendors can retrieve each vendor in parallel, while the comparison itself must wait for every retrieval to return. While treating a dependent subtask as independent lets a worker execute on incomplete

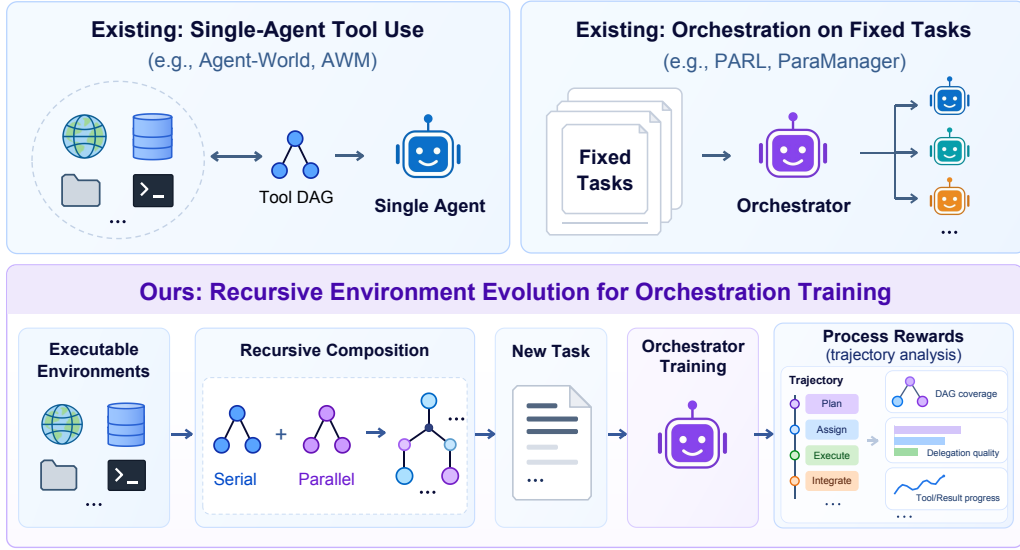


Figure 1: Existing environment synthesis builds executable tools and a verifier for one agent, and existing orchestration training consumes a fixed task set whose branch structure it does not control. MAW evolves the tasks of an executable environment by recursive environment improvement and trains an orchestrator on the resulting branches under schedule-aware process rewards.

information, treating an independent subtask as dependent serializes work that could have progressed concurrently. Throughout, the orchestrator never accesses a tool or a record itself. A single agent inspects one observation and corrects its next call. In contrast, the orchestrator observes only what workers report and therefore commits to a decomposition before any evidence exists. Therefore, an orchestrator learns this capability from multi-agent interaction and not from a collaboration procedure fixed in advance.

In most multi-agent systems, orchestration is still designed by hand, as in AutoGen and MetaGPT, where roles, routes, and execution order are fixed in a predefined workflow and the model operates within a collaboration structure that it does not determine. A recent line of work does train orchestration with reinforcement learning (Kimi Team, 2026; Yuan et al., 2026; Dang et al., 2025), which establishes that the capability is learnable. Scaling that training is limited by the environments available for learning. Specifically, a multi-agent environment offers a task whose subtasks can be distributed across agents, together with a verifier reporting how the team distributed them. Current environment synthesis provides neither property, as AWM (Wang et al., 2026), EnvScaler (Song et al., 2026), Agent-World (Dong et al., 2026) and ToolVerse (Zhou et al., 2026) build code-backed environments for a single agent that observes every tool return itself. A task synthesized this way is usually one chain of calls in which every step consumes the previous observation. In essence, single-agent environments and multi-agent environments differ in two ways. First, a single-agent task carries no controlled branch structure, thereby leaving an orchestrator with no scheduling decision to practice. Second, a single-agent task is scored only by terminal success. Under such a score, a team that partitions the work correctly and misses one integration step is indistinguishable from a team that never decomposes the request.

To this end, we introduce *Multi-Agent World* (MAW), an end-to-end framework for developing multi-agent orchestration. As shown in Figure 1, MAW couples multi-agent environment synthesis with reinforcement learning inside the synthesized environments. Specifically, to close the structural gap, MAW synthesizes multi-agent environments by *recursive environment improvement*. A serial operator links a subtask to the effect it consumes, and a parallel operator places independent subtasks on separate branches. Applying the two operators recursively grows a reference task graph, which semantic reconstruction then rewrites into one coherent user request. After the reference executes, replays from a fresh state, and passes semantic review, MAW admits the task into the training set. To close the supervision gap, MAW trains the orchestrator with *schedule-aware process rewards*. These rewards read the recorded execution of the whole team and score tool progress, answer coverage,

evidence delivery, and the quality of the schedule itself. MAW connects the two mechanisms through one record. Composition writes the branch structure into a task, and in the same step records which subtasks are verified as independent. This record distinguishes a correct parallel decomposition from one that succeeds by chance. Multi-agent environment synthesis therefore produces both the environment in which the team acts and the ground truth against which the orchestrator is rewarded.

We evaluate MAW on six agentic benchmarks covering multi-turn tool use, interactive task completion, executable multi-step tasks, long-horizon planning, broad information seeking, and real-world tool execution. Experimental results show that MAW improves task success by ___ over its single-agent initialization and by ___ over the same multi-agent runtime without an orchestration update, while shortening the execution critical path by ___. More importantly, the ablations show that the branches introduced by composition, and not the additional length of a composed task, account for the improvement. Our contributions are three-fold.

- **End-to-End Multi-Agent Framework:** To the best of our knowledge, MAW is the first framework that develops multi-agent orchestration end to end, synthesizing multi-agent environments and training the orchestrator inside them, where the construction record of a task also serves as the ground truth of its reward.
- **Recursive Environment Improvement and Schedule-Aware Rewards:** We propose recursive environment improvement, which converts single-agent environments into multi-agent environments by treating branch structure as a controlled construction variable, and schedule-aware process rewards, which make the quality of a decomposition visible in the gradient before any complete solution is sampled.
- **Strong Performance:** Experimental results on six agentic benchmarks show that MAW improves task success by ___ with an 8B backbone, and behavior audits show that the gain comes from fewer duplicated assignments and fewer missing integrations.

2 RELATED WORK

Environment Synthesis for Single-Agent Training. Training an agent with reinforcement learning requires environments that execute, hold state, and verify outcomes automatically, and a recent line of work builds such environments at scale for one agent. AWM synthesizes code-backed worlds with structured databases and rule-based checkers, thereby delivering a task with executable tools and a verifiable answer (Wang et al., 2026). EnvScaler separates the construction of an environment skeleton from the generation of scenarios and validators, which raises the yield of usable tasks (Song et al., 2026). Agent-World pairs environment discovery with a diagnostic loop that keeps tasks near the current ability of the policy (Dong et al., 2026), and ToolVerse follows tool dependencies to build long-horizon tasks (Zhou et al., 2026). Further work adapts existing environments through interface components or learns an environment-design role through self-play (Huang et al., 2026; Liu et al., 2026). Overall, these systems produce executable and automatically verified tasks for a single agent that observes every tool return itself. Such a task rarely contains subtasks that several agents can execute simultaneously, and its verifier reports only whether one agent finished.

Multi-Agent Collaboration and Orchestration Training. Multi-agent systems differ mainly in how the collaboration structure is decided. AutoGen organizes collaboration as conversations between agents (Wu et al., 2023), and MetaGPT encodes standardized operating procedures as role-based workflows (Hong et al., 2024). In both cases a human authors the structure and the structure stays fixed during execution. A second group searches the structure automatically, through optimizable agent graphs (Zhuge et al., 2024), workflow generation (Zhang et al., 2025b; Hu et al., 2025), and agentic supernets that select a topology per query (Zhang et al., 2025a). Although search improves the structure, the selected structure still fixes collaboration once execution starts. A third group updates the model that performs the orchestration. Representative methods include evolving orchestration for agent collaboration (Dang et al., 2025), parallel agent reinforcement learning that credits critical steps (Kimi Team, 2026), and unified agent and tool orchestration with parallel subtask decomposition (Yuan et al., 2026). Overall, learned orchestration is established as feasible. In these studies, the branch structure of the training distribution is inherited from existing benchmarks, thereby remaining neither controlled during construction nor available during reward computation.

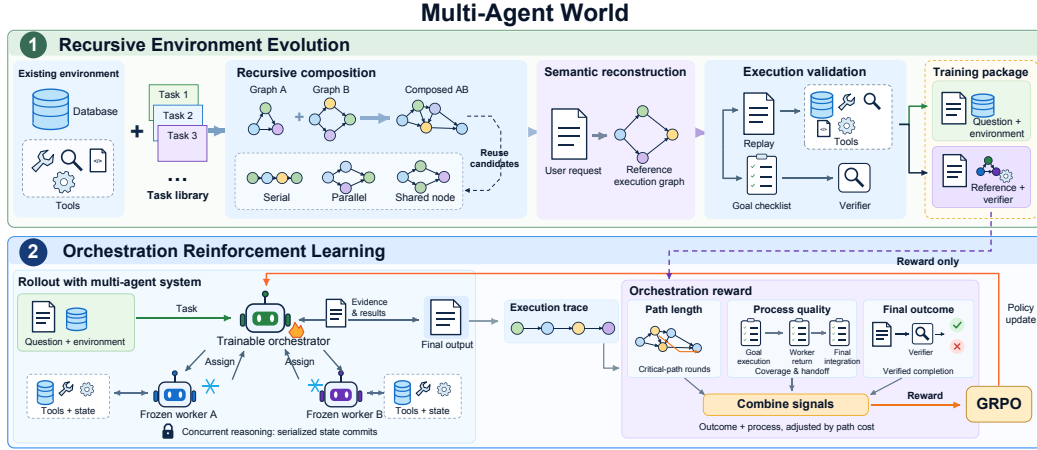


Figure 2: Overview of MAW. Recursive environment improvement composes the reference graphs of existing tasks, reconstructs each composed graph into one request, and admits it only after replay and verification. The resulting package supplies the interactive environment to the agent team and the hidden reference to the reward service, and only the orchestrator is updated.

Positioning of MAW in Multi-Agent Training. In this work, we focus on how the multi-agent environments an orchestrator trains in relate to the signal it trains under. Within backends that already execute and verify, our goal is to scale orchestration training. Unlike environment synthesis that targets a single agent, MAW treats branch structure as a construction variable and composes reference task graphs with serial and parallel operators. Dependencies and independent branches are thereby written into a task on purpose. Unlike orchestration learning that consumes a fixed task distribution and scores a rollout by terminal success, MAW adopts a more deliberate design by carrying the construction record into training. Specifically, the schedule of the agent team is scored against the independent subtasks that composition has already verified. We state what is new here precisely. Learned orchestration and synthesized environments have both been studied, and synthesizing an environment for the orchestration signal itself has not. In MAW, one construction supplies the task, its branches, and the ground truth that scores how those branches are used.

3 MULTI-AGENT WORLD

As shown in Figure 2, MAW develops multi-agent orchestration in two stages that share one construction record. The first stage turns existing single-agent backends into multi-agent environments, in which a task carries subtasks that several agents can execute simultaneously. The second stage trains an orchestrator inside those environments under a reward that reads how the agent team actually distributed the work.

3.1 FROM SINGLE-AGENT TO MULTI-AGENT ENVIRONMENTS

An agentic environment is written as $\mathcal{E} = (\mathcal{S}, \mathcal{U}, f_{\text{exec}}, s_0)$, where $\mathcal{S}$ is a database-backed state space, $\mathcal{U}$ is the tool interface, and s_0 is the initial state. The transition $f_{\text{exec}}(s, u) = (s', o)$ executes a tool call u and returns an observation o . Following the synthesis setting of AWM and Agent-World (Wang et al., 2026; Dong et al., 2026), a task in $\mathcal{E}$ is written as $x = (q, G, a^*, \Delta^*)$. Here q is the user request, a^* is the reference answer, and Δ^* is the required change of the database. The reference task graph $G = (V, A)$ has one node $v \in V$ per reference tool call, and an edge $(u, v) \in A$ means that v consumes an output or a state effect of u .

A multi-agent environment extends this setting with a delegation interface and an independence record. The interface $\mathcal{H}$ lets an orchestrator create workers and assign subtasks to them, which is what makes a team executable at all. The record Π closes the supervision gap of Section 1, since it names the branches against which a schedule can be scored, while the structural gap is closed by the composition that creates those branches. We write a multi-agent environment as $\mathcal{E}^{\text{ma}} = (\mathcal{S}, \mathcal{U}, f_{\text{exec}}, s_0, \mathcal{H})$ and a

multi-agent task as $x^{\text{ma}} = (q, G, a^*, \Delta^*, \Pi)$. Here $\Pi \subseteq V \times V$ contains the pairs of reference calls that are verified to be executable in either order. Orchestration training consumes this record, without which a verifier reports only whether the team produced the correct answer and leaves the correctness of a simultaneous execution unmeasured. Existing pipelines produce (q, G, a^*, Δ^*) for one agent and leave Π empty, which is why their tasks train tool use and not orchestration.

3.2 RECURSIVE ENVIRONMENT IMPROVEMENT

In this stage, we aim to turn existing single-agent backends into multi-agent environments without building new ones from scratch, which is challenging as branch structure cannot be requested from a generator directly. When a model is asked for a parallel task, it tends to produce two unrelated questions in one prompt, leaving an executable reference that is either trivial or inconsistent. MAW instead constructs branch structure at the level of reference graphs, where independence is a checkable property, and writes the graph as natural language only afterwards.

Recursive Composition. Before any natural language is generated, composition introduces a controlled amount of dependency and independence into a task. From a seed pool $\mathcal{C}$ of single-agent tasks in the same backend, MAW composes two of them into a candidate graph,

$$\tilde{G} = f_{\text{comp}}^z(G_1, G_2), \quad z \in \{\text{serial}, \text{parallel}\}, \quad (1)$$

where the serial operator adds an edge between a pair $(v_1, v_2) \in V_1 \times V_2$ whose output or state effect fills an argument of the other. In contrast, the parallel operator takes the disjoint union with no edge between the two sides and lets one answer node consume both. After composition, a candidate graph re-enters $\mathcal{C}$ and acquires further subtasks in later rounds, so mixed structure arises from repeated application. For each edge, MAW stores the operator that produced it, which later distinguishes a branch from an accidental ordering, and Appendix A gives a worked example of two rounds.

Semantic Reconstruction. A composed graph is not yet a request a user would write, as the two parents may refer to different entities or reporting formats. Reconstruction produces one coherent request and an executable reference plan, written as $(q, G) = f_{\text{recon}}(\tilde{G}, q_1, q_2, \mathcal{U})$, during which MAW aligns entities across the two sides, merges reads that return the same content, and inserts a lookup when the combined goal needs one. The request states the desired outcome and exposes neither the reference plan nor any hint about how many workers to use, thereby leaving the decomposition to the policy.

Execution-grounded Admission. As repeated composition can accumulate redundant calls or combine requirements that no single execution satisfies, MAW admits a task only when its reference survives execution and inspection. Executing the plan from a reset state gives $(a^*, \Delta^*) = f_{\text{exec}}^*(G, s_0)$, and MAW then replays it from a fresh state. MAW admits a task only when execution, replay, and an independent review all pass, written as $f_{\text{gate}} = f_{\text{run}} \cdot f_{\text{replay}} \cdot f_{\text{sem}} \in \{0, 1\}$ and specified in Appendix A. Admission is the step that keeps a composed candidate from entering training as an unsolvable or self-contradictory task.

Verified Independence Record. Although composition proposes that two subtasks lie on separate branches, the proposal is not yet verified, as reconstruction may introduce a shared entity that makes one side depend on the other. MAW verifies every proposed branch by execution before recording it,

$$\Pi = \{(v, v') \in V_r \times V_r : v \not\prec v', v' \not\prec v, f_{\text{perm}}(v, v') = 1\}, \quad (2)$$

where $V_r \subseteq V$ is the set of read-only calls and $v \prec v'$ denotes that v' is reachable from v in G . The test $f_{\text{perm}}(v, v')$ executes the reference plan again with the order of the two calls exchanged, returning one when the answer and the database effect are unchanged. Together with its resettable environment, its checker, and its hidden reference, this record converts a synthesized task into a multi-agent task. One construction therefore produces two things at once, namely the branches an orchestrator can act on and the ground truth against which its use of those branches is scored. Section 3.3 consumes the second of them.

3.3 MULTI-AGENT ORCHESTRATION LEARNING WITH SCHEDULE-AWARE PROCESS REWARDS

In this stage, we aim to teach an orchestrator to distribute work well, which terminal verification alone cannot express, as it collapses every unfinished team execution into the same value. A group of

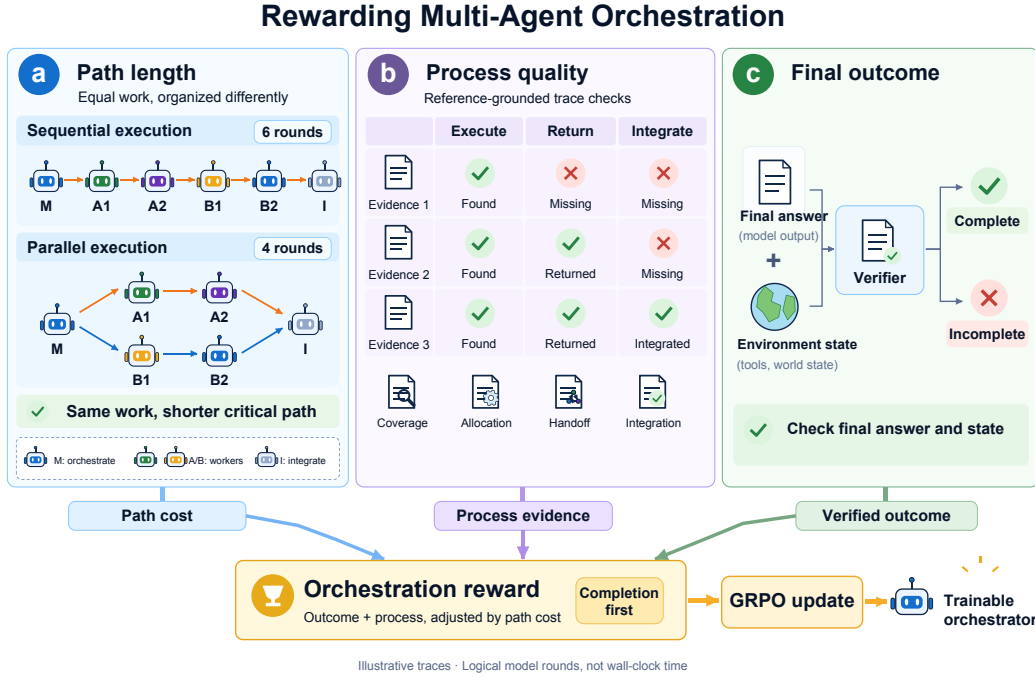


Figure 3: Three sources of the orchestration reward. Equal work organized in parallel shortens the critical path (a), reference-grounded checks separate execution, return, and integration for every piece of evidence (b), and the verifier accepts a trajectory only when the final answer and the environment state both hold (c).

rollouts that all fail produces identical returns, and their centered advantages vanish. The task then contributes no gradient during the stage where the policy fails most often. MAW addresses this by reading the recorded execution of the whole team and scoring the part of the work that was done correctly, including the schedule itself.

Delegation-only Interaction Harness. In the MAW harness, delegation is the only way to act, attributing every reward difference to orchestration and not to the orchestrator solving the task directly. Specifically, the orchestrator π_θ registers a worker with a role and a permitted tool subset through `create_subagent`. The orchestrator then assigns one subtask through `assign_task`, assigns several subtasks concurrently through `assign_tasks`, and ends the episode through `submit_answer`. During an episode, every worker follows the frozen policy π_ϕ and is the only actor that accesses $\mathcal{U}$. The runtime records each tool event with its arguments, observation, owning assignment, and causal parents, which is the record the reward reads, and Appendix C states the information boundary between the three participants.

Team-Level Outcome Verification. In a multi-agent setting, the outcome term determines whether the team completed the task by verifying the environment and not the report of any agent. For a trajectory τ with final answer a_τ and final state s_τ , the outcome term requires both an accepted answer and an accepted database state,

$$f_{\text{out}}(\tau) = V^{\text{ans}}(a_\tau, a^*) \cdot V^{\text{state}}(\Delta_\tau, \Delta^*) \in \{0, 1\}, \quad (3)$$

where V^{ans} and V^{state} are the answer and state checkers of the task, each returning one when the submitted result is accepted and zero when an episode ends without a submission. The state checker compares the required database mutations and rejects unexpected changes. Receiving only the account of a worker, the orchestrator can do no more than repeat it, so checking Δ_τ is what establishes that a record was created or updated and that unrelated rows were left untouched.

Schedule-aware Process Reward. As shown in Figure 3, the process term scores four properties of a team execution, namely the reference work completed, the requested answer present, the evidence

returned by workers, and the independent work scheduled as independent,

$$f_{\text{proc}}(\tau) = \alpha_t f_{\text{tool}}(\tau) + \alpha_a f_{\text{ans}}(\tau) + \alpha_d f_{\text{del}}(\tau) + \alpha_s f_{\text{sch}}(\tau), \quad (4)$$

where every term lies in $[0, 1]$ and the weights α are fixed once for all experiments (Table 11). The tool term f_{tool} is the fraction of reference calls in V matched once by a successful event with equivalent arguments and observation. The answer term f_{ans} gives recall-oriented partial credit over the top-level fields of the answer, with a precision factor that removes any gain from padding the submission (Appendix D). A team recovering most of the answer is thereby separated from one recovering none. The delivery term f_{del} is the fraction of assigned subtasks whose worker return carries the required evidence, which measures the handoff a single agent never performs. The schedule term is the part of the reward that no terminal verifier can express, and it reads the independence record produced by Eq. 2,

$$f_{\text{sch}}(\tau) = (1 - \delta_\tau) \cdot f_{\text{int}}(\tau) \cdot \frac{|\Pi_\tau|}{|\Pi|}, \quad (5)$$

The leading ratio is the fraction of verified independent pairs that the team actually ran as independent, where $\Pi_\tau \subseteq \Pi$ contains the pairs assigned to two different workers under assignment intervals that do not depend on each other. Two factors keep that ratio honest, since δ_τ discounts assignments that repeat a covered subtask and f_{int} discounts branch results that never reach the final answer. Eq. 5 is where synthesis and learning meet, as Π exists only because composition proposed a branch and a reordered execution confirmed it. A team is therefore credited for parallel work only on branches the environment has proven parallel, which is a judgement that terminal success cannot make. A task whose reference contains no verified independent pair falls back to a conservative progress score, and Appendix D specifies that branch.

Orchestrator-only Policy Optimization. Partial progress requires enough weight to distinguish failing rollouts and little enough to never compete with a verified success. The two terms are combined as

$$R(\tau) = f_{\text{out}}(\tau) + \lambda f_{\text{proc}}(\tau), \quad \lambda = 0.3, \quad (6)$$

which gives a successful team at least 1 and an unsuccessful team at most λ . Process credit thereby separates failed rollouts without ever outweighing success. For each task we sample K team trajectories from independently reset environments and compute the group-relative advantage $A_i = (R_i - \bar{R}) / (\text{std}(R_1, \dots, R_K) + \epsilon)$ of the i -th trajectory. The orchestrator is then updated with the clipped GRPO objective (Shao et al., 2024), written as

$$J(\theta) = \mathbb{E} \left[\frac{1}{K} \sum_{i=1}^K \frac{1}{|\mathcal{T}_i|} \sum_{t \in \mathcal{T}_i} \min(\rho_{it}(\theta) A_i, \text{clip}(\rho_{it}(\theta), 1 - \epsilon_l, 1 + \epsilon_h) A_i) \right], \quad (7)$$

where $\mathcal{T}_i$ contains only the tokens generated by the orchestrator and ρ_{it} is the ratio between the current and the behavior policy on that token. Worker generations and tool observations enter the context of the orchestrator and stay out of $\mathcal{T}_i$. Since π_ϕ is never updated, every measured change comes from orchestration and not from worker quality.

4 EXPERIMENTS

4.1 BENCHMARKS

We evaluate on six agentic benchmarks that together cover the decisions an orchestrator makes. **BFCL Multi-Turn** (Patil et al., 2025), **τ^2 -Bench** (Barres et al., 2025), and **ACEBench-Agent** (Chen et al., 2025) measure tool use across turns, interactive completion under a simulated user, and executable multi-step tasks, and they are dominated by dependent call sequences. In contrast, **DeepPlanning** (Zhang et al., 2026), **WideSearch** (Wong et al., 2025), and **MCPMark** (Wu et al., 2025) measure long-horizon planning under verifiable constraints, broad information seeking that fills one structured table, and real tool execution through Model Context Protocol services, and their requests carry evidence that can be gathered along several branches. Together, the two groups measure orchestration from complementary directions, and Appendix E records the evaluation protocol.

4.2 BASELINES

Under one runtime and one budget, we compare MAW with four groups of systems. *Frontier models* run as single agents under the same tool access and indicate the headroom of each benchmark, and

Table 1: Performance on six agentic benchmarks. Every trained system uses a Qwen3-8B backbone, and Qwen3-14B is listed as an untrained reference only. Metrics are multi-turn accuracy, task success, case accuracy, item F1, and pass rate, all in %. **Shaded** marks our method and **bold** marks the best 8B result per column.

Model	BFCL MT Accuracy	τ^2 -Bench Success	ACEBench Success	DeepPlanning Case Acc.	WideSearch Item F1	MCPMark Pass Rate
<i>Frontier Models</i>						
DeepSeek-V3.2	37.39	80.37	80.62			
GPT-4.1	38.88	54.67	82.92			
Kimi-K2-Instruct	50.63	64.30	79.17			
<i>Open-Source Backbones</i>						
Qwen3-14B						
Qwen3-8B	28.88	27.87	46.51			
<i>Single-Agent Environment Synthesis</i>						
AWM						
EnvScaler						
ToolVerse	37.50	32.37	61.66			
<i>Orchestration Training (8B)</i>						
Single-agent (init)						
Puppeteer						
ParaManager						
Multi-agent (init)						
MAW (ours)						

open-source backbones are the untrained Qwen3-8B and Qwen3-14B models (Qwen Team, 2025), of which Qwen3-8B initializes every trained system. *Single-agent environment synthesis* covers AWM (Wang et al., 2026), EnvScaler (Song et al., 2026), and ToolVerse (Zhou et al., 2026). Each trains the same backbone on its released resources under a common outcome-based recipe, testing the utility of a training resource without reproducing every optimization detail of the original system. *Orchestration training* covers Puppeteer (Dang et al., 2025) and ParaManager (Yuan et al., 2026), together with two settings of our own system. *Single-agent (init)* is the trained single-agent checkpoint that initializes MAW, executed without workers, and it is the reference point for every gain we report. *Multi-agent (init)* runs the MAW runtime with that same checkpoint as an orchestrator and applies no orchestration update, which isolates the effect of the execution architecture alone.

4.3 SETUP

We build MAW on Qwen3-8B and initialize the orchestrator and the workers from the same single-agent checkpoint, thereby attributing any measured difference to orchestration training. Multi-agent environment synthesis runs over publicly released tool backends and admits 2,581 tasks across 539 environments, which we use for orchestration training. In every comparison, the generator and the semantic reviewer use the same frontier model. With the worker policy frozen, we train the orchestrator with GRPO under Eq. 6 and sample $K = 8$ team trajectories per task from independently reset environments. Inference uses the vLLM engine, and all training and evaluation run on NVIDIA A800 GPUs. Every benchmark applies one shared token budget, timeout, worker-concurrency cap, and decoding configuration across compared systems, with failed and timed-out episodes kept in the denominator.

4.4 MAIN RESULTS

Capability on Dependent Tool Use. We first evaluate MAW on BFCL Multi-Turn, τ^2 -Bench, and ACEBench-Agent to assess its *dependent tool-use capability*. In this group, a request expands into a chain of calls that leaves little to schedule, where each call depends on the effect of the earlier ones. As shown in Table 1, MAW reaches ____ on BFCL Multi-Turn and ____ on τ^2 -Bench, exceeding every

Table 2: Ablation on multi-agent environment synthesis. All rows share the backends, the generator, the admission checks, and the outcome-only recipe. Yield is admitted tasks per fixed generation budget, $|\Pi|$ is the number of verified independent subtask pairs per admitted task, and success is held-out task success at an equal number of admitted tasks.

Training-task source	Yield (%) $\uparrow$	Subtasks / task	$ \Pi $ / task	Success (%) $\uparrow$
Original seed tasks				
Direct multi-goal synthesis				
One-round composition				
Recursive serial-only				
Recursive parallel-only				
Recursive environment improvement (ours)				

8B system in the comparison. On ACEBench-Agent it stays within ____ of the strongest single-agent synthesis baseline. Across the three benchmarks, MAW gains most on settings where a team can verify before committing, as a withheld function or a user-modified record is recovered by assigning a verification subtask. However, multi-agent (init) loses accuracy on the same settings, which stems from assignments that no worker can satisfy. Overall, the results on this group show that MAW improves the tool use a single agent already performs, without degrading the dependent call chains of the backbone.

Capability on Branch-rich Long-horizon Tasks. We then evaluate MAW on DeepPlanning, WideSearch, and MCPMark to assess its *capability on separable work*. In this group, constraint checks, retrievals, and service calls proceed along several branches before they are combined. As shown in Table 1, MAW raises case accuracy by ____ on DeepPlanning, reaches ____ item F1 on WideSearch, and improves the MCPMark pass rate by _____. Notably, multi-agent (init) falls below single-agent (init) on MCPMark, where an untrained orchestrator increases cost and does not improve success. Recursive environment improvement produces the same kind of branch structure during synthesis, although over entirely different backends. The orchestrator has thereby learned to assign independent checks to separate workers and to collect their results. The audit in Section 5.2 attributes the WideSearch improvement to fewer missing rows and not to more retrieval calls. MAW also reaches these results at a shorter critical path than multi-agent (init), which Section 5.4 reports alongside the token cost. Overall, the strong results on this group highlight the transfer of synthesized branch structure to long-horizon tasks that the training distribution never contained.

5 ANALYSES

5.1 ABLATION STUDY ON RECURSIVE ENVIRONMENT IMPROVEMENT

To evaluate the effect of branch structure on the usefulness of a synthesized task, we compare six strategies for producing training tasks. Across all rows, we keep the backends, the generator, the admission checks, and the outcome-only training recipe fixed. The sources range from the original seeds and direct synthesis of a multi-goal request to one round of composition, recursion restricted to one operator, and full recursive environment improvement. Measuring admitted tasks per generation budget alongside held-out success at an equal number of admitted tasks separates producing more usable data from producing more useful data.

As shown in Table 2, direct synthesis admits far fewer tasks than composition at the same budget. This gap stems from writing two goals into one prompt before any execution, which frequently produces a reference that fails or contradicts itself. In contrast, composition checks compatibility on reference graphs before any language is generated. Under serial-only recursion, the admitted references are longer and carry almost no verified independent pairs. Their held-out success also improves by less than that of full recursion, showing that task length alone does not train orchestration. Under parallel-only recursion, the admitted tasks carry shallow dependencies and degrade on tasks that require waiting for a prerequisite. Among the six sources, only full recursion admits tasks that carry a non-trivial Π together with real dependencies, highlighting the effectiveness of applying the two operators recursively. At the level of the distribution, reference tool calls grow by 19.6%

Table 3: Ablation on the schedule-aware process reward, with the behavior audit it targets. Each row sets one coefficient of Eq. 4 to zero without redistributing the weight. Duplicate is excess assignment ownership per attempted subtask, Handoff counts workers that omit obtained evidence, Integr. counts branch results that are available and absent from the final answer, and C is the number of logical model rounds on the critical path.

Training reward	Success $\uparrow$	Goal cov. $\uparrow$	Delivery $\uparrow$	Duplicate $\downarrow$	Handoff $\downarrow$	Integr. $\downarrow$	C $\downarrow$
Multi-agent (init)							
Outcome only							
w/o tool progress							
w/o answer coverage							
w/o evidence delivery							
w/o schedule term							
Full reward (ours)							

relative to the two parents of each task, while top-level answer fields grow by 57.3% (Appendix B). A composed task thereby asks for more results that can be produced independently, which is the property an orchestrator can exploit.

5.2 ABLATION STUDY ON SCHEDULE-AWARE PROCESS REWARDS

To verify the effectiveness of each term of Eq. 4, we conduct an ablation study that removes one term at a time, setting its coefficient to zero without redistributing the weight. We also train an outcome-only setting. All settings share the task set, the initialization, the frozen workers, the training budget, and the evaluation instances, thereby isolating the reward as the only difference. The schedule term applies to the ____ of training tasks that carry a verified independent pair, and the remaining tasks use the fallback score of Appendix D. Besides task success, we audit held-out tasks whose reference annotations link every assignment to a subtask, reporting goal coverage, evidence delivery, duplicate allocation, handoff failure, and integration failure. Together, these measurements distinguish a change in behavior from a change in score.

As shown in Table 3, the outcome-only setting is the weakest on every behavioral measurement. Its training curve also stays flat during the early stage where most groups of rollouts fail together. Such a stage is the vanishing-advantage regime that motivates the process term. Removing tool progress lowers goal coverage and leaves the schedule almost unchanged, while removing evidence delivery raises handoff failures as success falls more slowly. The two terms thus act on different parts of a team execution. Among the four components, removing the schedule term produces the most informative result, where success drops less than goal coverage would suggest, the critical path grows back toward multi-agent (init), and integration failures return to the same level. The team continues to solve the task and stops distributing it. The results demonstrate the advantage of scoring the schedule against verified independent subtasks, so the schedule term is not redundant with the other three. Notably, the full reward is the only configuration that lowers duplicate allocation, handoff failure, and integration failure simultaneously. Under the full reward, MAW thus improves the benchmarks of Section 4 through the intended orchestration behavior and not through longer executions.

5.3 EFFECT OF PROCESS REWARDS ON TRAINING DYNAMICS

To evaluate the mechanism we claim for the process term, we record the fraction of task groups with a non-zero centered advantage, which measures how much of each batch is usable. Over the first ____ updates, the outcome-only setting retains only ____ of its task groups, since a task that no rollout solves returns the same value for every trajectory. The full reward keeps that fraction high from the first update, as the four process terms differ between two failing teams that made different amounts of progress, and the success curves separate after the point where the two settings diverge.

Table 4: Effect of the execution architecture, the training source, and the training reward. Settings A to D use an outcome-only reward at an equal training-set size, E adds the schedule-aware process reward, and F reuses the weights of E with concurrent assignments serialized at inference. C is the number of logical model rounds on the critical path, and tokens are summed over the orchestrator and every worker.

	Training source	Execution	Reward	Success (%) $\uparrow$	C $\downarrow$	Tokens (k) $\downarrow$
A	AWM	Single-agent	Outcome			
B	AWM	Multi-agent	Outcome			
C	MAW	Single-agent	Outcome			
D	MAW	Multi-agent	Outcome			
E	MAW	Multi-agent	Outcome + process			
F	MAW	Multi-agent, serialized	Outcome + process			

Table 5: Gain of MAW over single-agent (init) across branch density, measured on held-out MAW tasks in three equal-size groups. Branch density is the number of verified independent subtask pairs per task, and the last column is the share of assignments issued concurrently.

Branch density group	Single-agent (%)	MAW (%)	Gain $\uparrow$	Concurrent assign. (%)
Low				
Medium				
High				

5.4 EFFECT OF THE ARCHITECTURE, THE TRAINING SOURCE, AND THE REWARD

To investigate whether the improvement of MAW comes from running more agents or from training on more data, we conduct a controlled study that separates three factors. Among the six settings, A to D train with an outcome-only reward on an equal number of tasks. A and C are single-agent, B and D are multi-agent, A and B use AWM data, and C and D use MAW data. Setting E adds the schedule-aware process reward to D.

As shown in Table 4, moving from A to B raises the critical path and does not improve success. Adding a multi-agent runtime to a policy that was never trained to orchestrate mostly adds coordination cost. Moving from A to C improves single-agent success as well, as composed tasks are longer and more demanding for one agent, which is the part of the MAW data effect that is not specific to orchestration. Among these settings, B versus D and D versus E isolate the two components of MAW. MAW data provide independent subtasks to schedule, the schedule-aware reward trains the policy to use them, and neither factor alone reaches the success of E. In terms of execution cost, setting B requires a longer critical path and more tokens than single-agent execution and does not improve success, while setting E returns the critical path below multi-agent (init). Setting F isolates the concurrency itself, reusing the weights of E and executing the requested assignments in order, which preserves their content and removes only the overlap. Success stays close to E while the critical path grows, so MAW improves quality through the decomposition and reduces cost through the concurrency. Overall, the grid shows that the architecture, the data, and the reward contribute in that order of dependence. In particular, the proposed reward converts branch structure into measurable behavior.

5.5 ANALYSIS OF GAINS ACROSS BRANCH DENSITY

To explore how the improvement varies with task structure, we conduct a stratified analysis over the held-out tasks by branch density. Branch density is the number of verified independent subtask pairs a task contains, and within each group we measure the gain of MAW over the single-agent initialization alongside the share of concurrent assignments.

As shown in Table 5, the gain grows with branch density. In the same direction, the orchestrator issues concurrent assignments more often, and on the low-density group the two systems stay close, since there is no scheduling opportunity. The gain and the concurrent assignments therefore grow

together, which highlights the advantage of writing branch structure into the training tasks, as the policy exploits the property that composition introduces.

Further Analyses. Appendix B.1 reports the sensitivity of MAW to the process coefficient and the schedule weight, the transfer of the trained orchestrator to a different worker backbone, and a successful and an unsuccessful episode on the same branch-heavy task.

6 CONCLUSION

We presented MAW, an end-to-end framework that develops multi-agent orchestration by synthesizing multi-agent environments and training an orchestrator inside them. Recursive environment improvement composes the reference graphs of single-agent tasks, reconstructs each composed graph into one coherent request, and admits it only after execution, replay, and semantic review. The same construction verifies which subtasks are independent, which is what a schedule-aware process reward needs in order to score the whole team. MAW thereby derives the environment in which the agents act and the ground truth that scores them from one construction. Experiments on six agentic benchmarks show that this design improves task success and shortens the execution critical path.

The framework has four limitations. Reward matching is anchored on a reference execution, which gives a correct alternative solution less process credit than a reference-matching one. The schedule term also covers only the tasks whose independence survives a reordered execution. Measured in logical model rounds, the critical path omits tool latency, so we establish a shorter critical path and not a proportional reduction in wall-clock time. Requiring at least one delegation, the current interface never teaches the orchestrator when to solve a task alone, and evidence from one model scale with frozen workers leaves jointly trained teams for future work.

REFERENCES

- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment, 2025. URL <https://arxiv.org/abs/2506.07982>.
- Chen Chen, Xinlong Hao, Weiwen Liu, Xu Huang, Xingshan Zeng, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Yuefeng Huang, et al. ACEBench: Who wins the match point in tool usage?, 2025. URL <https://arxiv.org/abs/2501.12851>.
- Yufan Dang, Chen Qian, et al. Multi-agent collaboration via evolving orchestration. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL <https://arxiv.org/abs/2505.19591>.
- Guanting Dong, Juntong Lu, Junjie Huang, Wanjuan Zhong, Longxiang Liu, Shijue Huang, Zhenyu Li, Yang Zhao, Xiaoshuai Song, Xiaoxi Li, et al. Agent-world: Scaling real-world environment synthesis for evolving general agent intelligence. *arXiv preprint arXiv:2604.18292*, 2026. URL <https://arxiv.org/abs/2604.18292>.
- Jeremy Hadfield, Barry Zhang, Kenneth Lien, Florian Scholz, Jeremy Fox, and Daniel Ford. How we built our multi-agent research system. Anthropic Engineering Blog, June 2025. URL <https://www.anthropic.com/engineering/multi-agent-research-system>.
- Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiaowu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for a multi-agent collaborative framework, 2024. URL <https://arxiv.org/abs/2308.00352>.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems, 2025. URL <https://arxiv.org/abs/2408.08435>.
- Chengsong Huang, Zifeng Wang, Rujun Han, Jun Yan, Yanfei Chen, Zoey CuiZhu, Ke Jiang, Peng Xia, Han Yu, Yufan Zhuang, Yifei Ming, Jiaqi Pan, Bhavana Dalvi Mishra, Jiaxin Huang, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. EnvHarness: Awakening static worlds for agent learning, 2026. URL <https://arxiv.org/abs/2608.19880>.

- Kimi Team. Kimi K2.5: Visual agentic intelligence, 2026. URL <https://arxiv.org/abs/2602.02276>.
- Bo Liu, Simon Yu, Yiding Jiang, Ao Qu, Andrew Zhao, Zichen Liu, Junsu Kim, Zijian Zhou, Seungone Kim, Tongzheng Ren, Mickel Liu, Hanfei Yu, Zhaorun Chen, Weiyan Shi, Paul Pu Liang, Luke Zettlemoyer, Yejin Choi, and Natasha Jaques. SPADE: Self-play in adaptive synthetic executable environments, 2026. URL <https://arxiv.org/abs/2608.19197>.
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=2GmDdhBdDk>.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=dHng200Jjr>.
- Qwen Team. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023. URL <https://openreview.net/forum?id=Yacmpz84TH>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Yang Wu, et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Xiaoshuai Song, Haoyu Chang, Guanting Dong, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. EnvScaler: Scaling tool-interactive environments for LLM agent via programmatic synthesis, 2026. URL <https://arxiv.org/abs/2601.05808>.
- Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent world model: Infinity synthetic environments for agentic reinforcement learning, 2026. URL <https://arxiv.org/abs/2602.10090>.
- Ryan Wong, Jiawei Wang, et al. WideSearch: Benchmarking agentic broad info-seeking, 2025. URL <https://arxiv.org/abs/2508.07999>.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023. URL <https://arxiv.org/abs/2308.08155>.
- Zijian Wu et al. MCPMark: A benchmark for stress-testing realistic and comprehensive MCP use, 2025. URL <https://arxiv.org/abs/2509.24002>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- Wenzhen Yuan et al. Small model as master orchestrator: Learning unified agent-tool orchestration with parallel subtask decomposition, 2026. URL <https://arxiv.org/abs/2604.17009>.
- Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. Multi-agent architecture search via agentic supernet, 2025a. URL <https://arxiv.org/abs/2502.04180>.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating agentic workflow generation. In *International Conference on Learning Representations (ICLR)*, 2025b. URL <https://arxiv.org/abs/2410.10762>.

Yinger Zhang, Shutong Jiang, Renhao Li, Jianhong Tu, Yang Su, Lianghao Deng, Xudong Guo, Chenxu Lv, and Junyang Lin. DeepPlanning: Benchmarking long-horizon agentic planning with verifiable constraints, 2026. URL <https://arxiv.org/abs/2601.18137>.

Shuaiyu Zhou et al. ToolVerse: Unlocking massive environments and long-horizon tasks for agentic reinforcement learning, 2026. URL <https://arxiv.org/abs/2607.15660>.

Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. Language agents as optimizable graphs, 2024. URL <https://arxiv.org/abs/2402.16823>.

A MULTI-AGENT ENVIRONMENT SYNTHESIS

A.1 ADMISSION CHECKS

Table 6 lists the checks behind the admission decision f_{gate} of Section 3.2. A candidate that fails any check is returned to the generator with the failing evidence, and a candidate that cannot be repaired within the generation budget is discarded. Every attempt starts from a reset environment, so an admitted task never depends on state left by an earlier attempt.

Table 6: Admission checks and their acceptance conditions. The first three checks are executed by code, and the fourth is a semantic review that reads the request together with the recorded execution evidence.

Check	Acceptance condition
Plan validity	Allowed tools, valid arguments, unique call identifiers, and references that resolve to earlier observations.
Graph consistency	One node per reference call plus one answer node, acyclic forward edges consistent with the observed execution, and state-order edges supported by an observed state change.
Execution and replay	The reference plan executes, the checker accepts the produced answer and database effect, the checker rejects corrupted answers and state, and a replay from a fresh state reproduces both without unexpected mutations.
Semantic review	The request is fulfilled, the answer is complete, both parent goals survive, the operations form one coherent task, the state claims agree with the database, and every repeated call serves a distinct purpose.

A.2 INDEPENDENCE VERIFICATION

Eq. 2 records a pair of reference calls as independent only when the graph contains no path between them in either direction, both calls are non-mutating, and the reference plan produces the same answer and the same database effect when the two calls are executed in the opposite order. The permutation test is run inside a reset environment, and a pair that fails it is dropped from Π even when composition proposed it as a parallel branch. Tasks whose reference contains mutating operations on shared records therefore contribute a small Π , and the schedule term of Eq. 5 is computed over the recorded pairs only. Section 5.2 reports the fraction of training tasks that carry a verified independent pair.

A.3 SYNTHESIS CASES

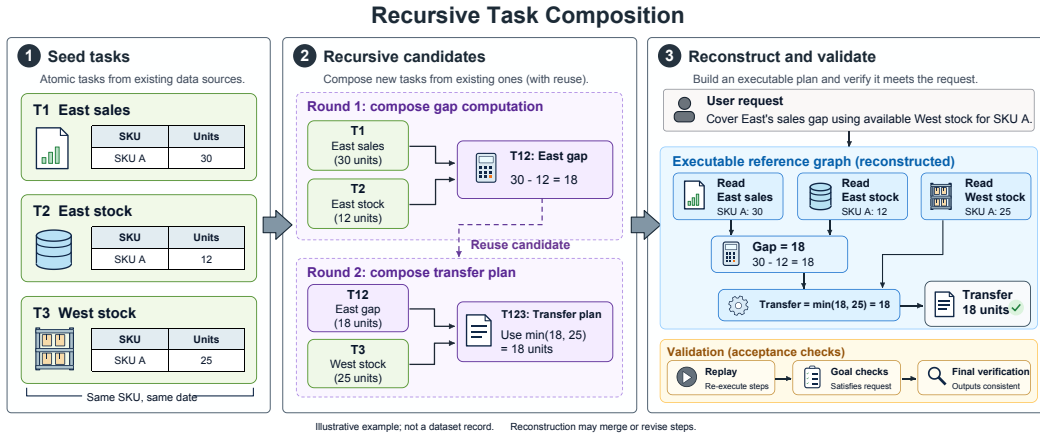


Figure 4: Recursive composition on an illustrative task family. Two rounds of the serial operator turn three atomic seeds into a transfer plan, and reconstruction rebuilds the composed candidate as one executable reference graph that replay and goal checks then validate.

Figure 4 first shows the mechanism on an illustrative task family. The two cases below show how composition combines parent goals and how reconstruction turns a composed graph into an executable task. Parent goals and observations are summarized for readability, and the generated request and the reference answer are reproduced in full.

Case 1 Parallel branches over a control plane

Environment. Control planes with routes and services, and groups of member control planes.

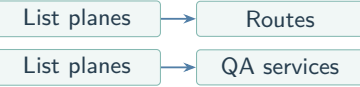
Parent tasks and composed graph

Reference union: 8 calls

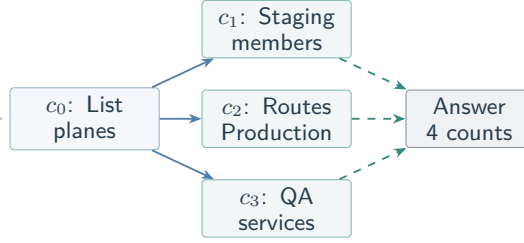
Parent A: Staging members + QA total



Parent B: Production routes + QA gRPC



Reconstructed DAG: 4 calls



→ data dependency

- - -> answer evidence

The two parent graphs form an eight-call reference union. Reconstruction merges the four discovery calls into one and the two service queries into one, and all four requested counts survive.

Generated request

Find the control plane group named 'Staging' and count how many member control planes belong to that group. Then find the control plane named 'Production' with cluster type 'k8s' and count how many routes it has. Next, find the control plane named 'QA' with cluster type 'hybrid' in the 'ap-south' region and count both how many services are deployed to it and how many of those services use the 'grpc' protocol. Return the member count for the Staging group, the route count for Production, the total service count for QA, and the gRPC service count for QA.

Reference execution

Tool call	Observation
<code>c0 list_control_planes()</code>	Staging group 2, Production 1, QA 5.
<code>c1 list_control_plane_group_memberships(groupId=2)</code>	2 member planes.
<code>c2 list_routes(controlPlaneId=1)</code>	30 routes.
<code>c3 list_services(controlPlaneId=5)</code>	10 services, 6 use gRPC.

Reference answer and record

```

{
  "staging_group_member_count": 2,
  "production_route_count": 30,
  "qa_total_service_count": 10,
  "qa_grpc_service_count": 6
}
  
```

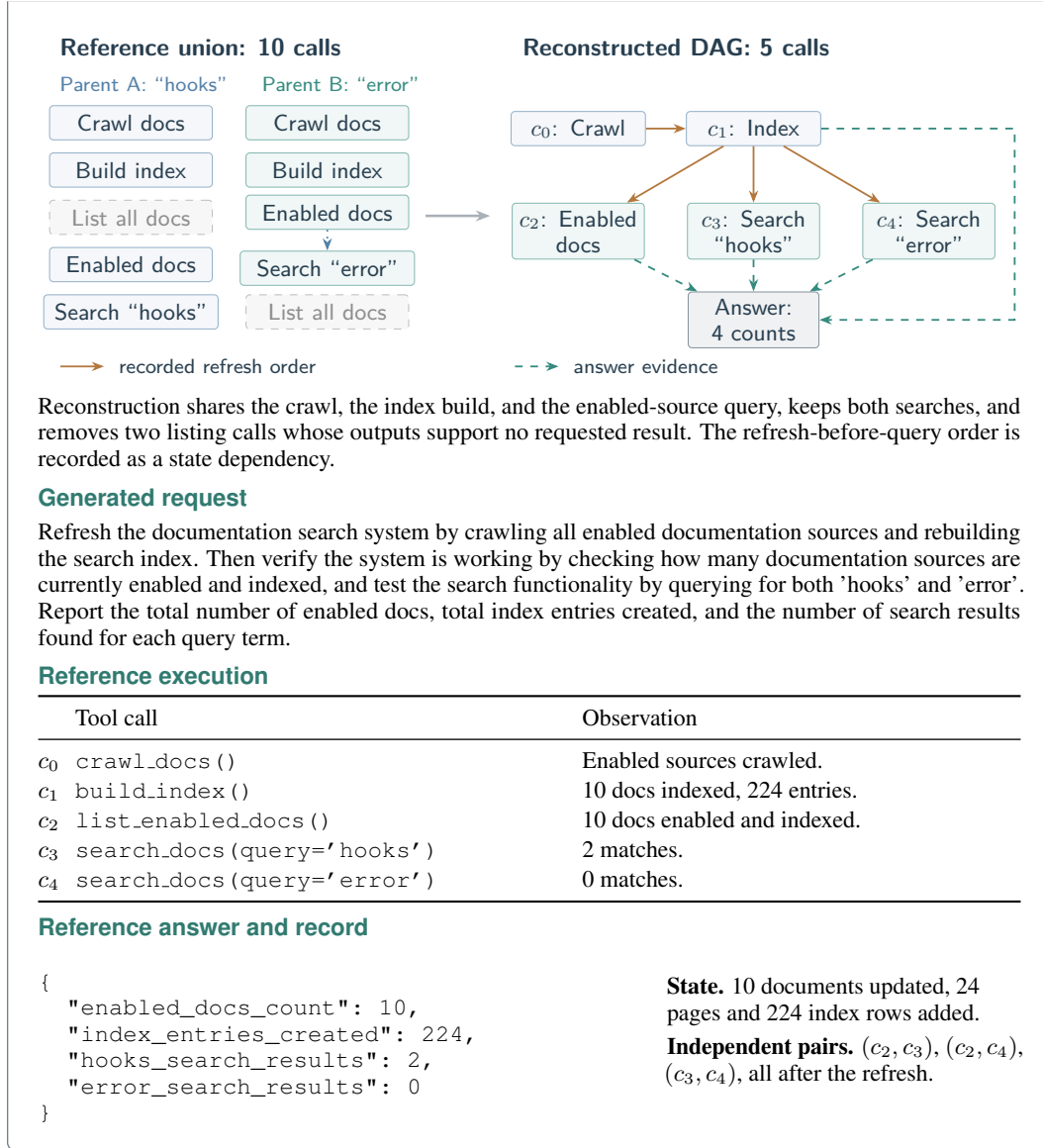
State. Unchanged.
Independent pairs. (c_1, c_2) , (c_1, c_3) , (c_2, c_3) , all confirmed by reordered execution.

Case 2 Serial dependency with pruned calls

Environment. Documentation sources with crawl, indexing, and search operations.

Parent tasks and composed graph

Parent A refreshes the sources, counts enabled documents, and searches for "hooks". Parent B refreshes the same sources, reports enabled documents and index size, and searches for "error".



B DATASET PROFILE AND ADDITIONAL ANALYSES

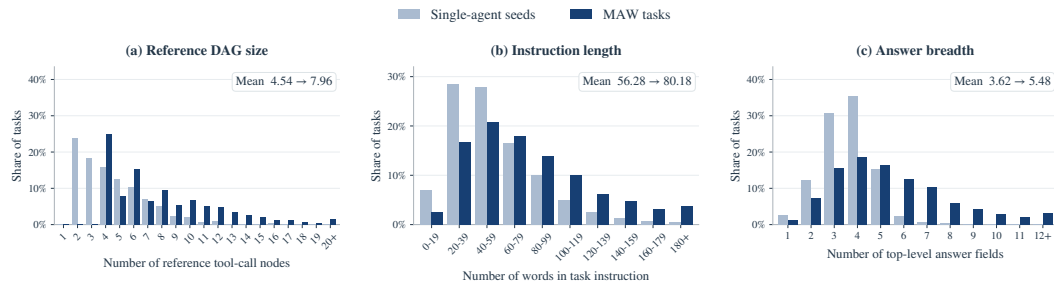


Figure 5: Reference structure of the seed pool and of the admitted MAW tasks over 10,860 seeds and 2,581 admitted tasks in 539 environments. Composition grows the number of answer fields far more than the number of tool calls, which indicates that an admitted task asks for more separable results and not simply a longer call chain.

Table 7: Sensitivity of MAW to the process coefficient λ and the schedule weight α_s . All rows share the task set, the initialization, and the training budget. The setting used throughout the paper is shaded.

λ	Success (%) $\uparrow$	Goal cov. (%) $\uparrow$	$C \downarrow$	α_s	Success (%) $\uparrow$	Goal cov. (%) $\uparrow$	$C \downarrow$
0.0				0.00			
0.1				0.15			
0.3				0.30			
0.5				0.45			
1.0				0.60			

Table 8: Generalization of the trained orchestrator across worker backbones. The orchestrator is trained once with frozen Qwen3-8B workers and then paired with different workers at inference without further updates.

Orchestrator	Workers	Success (%) $\uparrow$	$C \downarrow$	Delivery $\uparrow$
Single-agent (init)	Qwen3-8B			
MAW	Qwen3-8B			
Single-agent (init)	Qwen3-14B			
MAW	Qwen3-14B			

As shown in Figure 5, reference tool calls grow by 19.6% relative to the two parents of each admitted task, distinct tools by 3.9%, and top-level answer fields by 57.3%. The matched baseline of a task is the mean of its two parents, and a parent reused by several descendants contributes once per descendant.

B.1 SENSITIVITY, TRANSFER, AND QUALITATIVE INSPECTION

Effect of the Process Coefficient and the Schedule Weight. To verify that the reported gains do not depend on a narrow choice of reward weights, we sweep the process coefficient λ of Eq. 6 and the schedule weight α_s of Eq. 4, keeping every other setting fixed. The sweep covers the value that removes the process term entirely and values that raise it until process credit approaches the scale of a verified success.

As shown in Table 7, success is stable across the middle of both ranges and degrades only at the two ends. A very small λ reproduces the outcome-only behavior, and a very large λ lets a team collect process credit without finishing the task, which raises goal coverage while success falls. The schedule weight behaves in the same way, where removing it returns the critical path to the untrained level and raising it too far encourages splitting work that is cheaper to keep together. The plateau in the middle indicates that the values in Table 11 were not selected from a narrow optimum.

Generalizability across Worker Backbones. To verify that orchestration training produces a transferable policy and not a policy fitted to one particular worker, we pair the orchestrator trained with frozen Qwen3-8B workers with workers of a different capability at inference and apply no further update. We keep the interface, the prompts, the budget, and the evaluation instances unchanged, so the only difference is the worker backbone.

As shown in Table 8, MAW improves over the untrained orchestrator under both worker backbones, and the gain in delivery is larger with the weaker workers, where reassignment after an incomplete return matters more. The critical path shortens under both settings, which indicates that the learned schedule depends on the structure of the request and not on the identity of the worker. Overall, the results show that the orchestration policy learned in MAW environments carries to teams it was not trained with, which is the property an orchestrator needs in deployment.

Case Study. To make the learned behavior concrete, we inspect one successful and one unsuccessful episode from the branch-heavy slice of the held-out set, selected by a fixed sampling rule over the

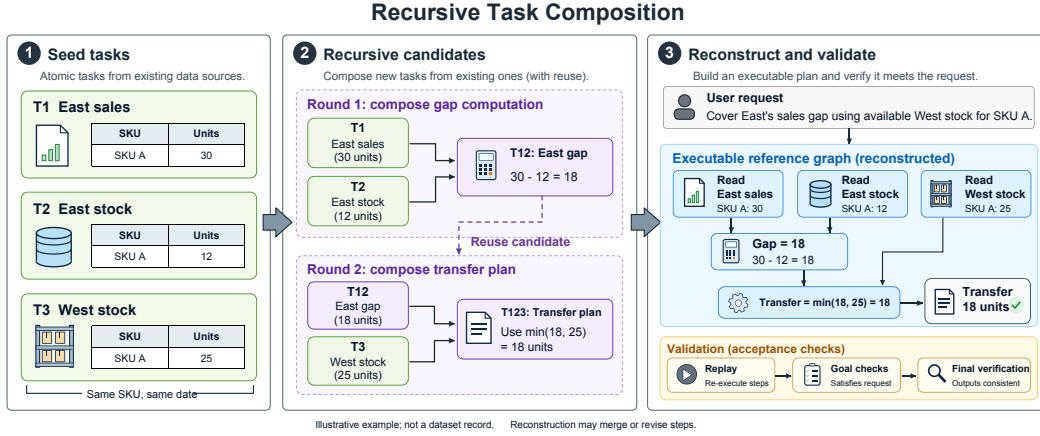


Figure 6: A successful and an unsuccessful episode on the same branch-heavy task. The successful team waits for the discovery result, assigns three independent lookups together, and integrates all of them. The unsuccessful team recovers the same evidence and omits one requested field, which the schedule term penalizes through its integration factor and the outcome term cannot distinguish from a complete failure.

structure of the reference. In the successful episode, the orchestrator issues one discovery assignment, waits for the identifiers it returns, then assigns three independent lookups in a single concurrent call and merges their results, which matches the dependency and branch pattern that composition wrote into the task. In the unsuccessful episode, the orchestrator recovers all the branch evidence and drops one of the requested fields when it writes the final answer, which is the integration failure that Eq. 5 penalizes and the outcome term alone cannot distinguish from a complete failure. As shown in Figure 6, the two episodes differ only in the final integration step, and the process reward separates them while the outcome term does not. Appendix A provides the full traces, the reference graphs, and the corresponding reward components.

C MULTI-AGENT RUNTIME AND INFORMATION BOUNDARY

Table 9 lists the four actions of the delegation-only harness.

Table 9: Orchestration interface. A worker is registered with a role and a permitted tool subset, a single assignment is synchronous, and a batch assignment launches workers concurrently and returns when their results join.

Action	Required information and effect
<code>create_subagent</code>	Name, system prompt, and allowed tool names, which registers a worker that follows the frozen policy.
<code>assign_task</code>	Worker name and subtask prompt, which executes one assignment and returns its result.
<code>assign_tasks</code>	A list of assignments, which permits concurrent worker inference and joins the returns.
<code>submit_answer</code>	The final answer in the required format, which triggers completion and verification.

Worker reasoning can overlap, and tool operations on the database commit under a transaction lock, so a concurrent assignment never produces an interleaving that the environment cannot reproduce. Table 10 states which information each participant can access during an episode. The reference plan, the expected answer, and the reward components are available only to the reward service, and no reward report is returned to the policy as an observation. A leakage audit inspects the rendered requests and tool descriptions of the training and evaluation manifests for reference answers, serialized reference graphs, and verifier diagnostics, and tasks descending from the same seed family are grouped before any split.

Table 10: Information boundary during an episode. Reward reports are never returned as observations, so the policy cannot read its own process score.

Information	Orchestrator	Worker	Reward service
User request	Yes	Assigned content	Yes
Allowed tool descriptions	Yes	Permitted subset	Yes
Tool and worker returns	Returned content	Own context	Logged content
Reference plan and expected answer	No	No	Yes
Independence record Π	No	No	Yes
Reward component scores	No	No	Yes

Critical-path Reconstruction. Execution cost is a diagnostic and is not part of Eq. 6. Let $w(v) = 1$ for each orchestrator or worker generation event and zero otherwise. Retaining only causal parents, the depth of an event and the cost of an episode are

$$d(v) = w(v) + \max_{u \in \text{pa}(v)} d(u), \quad C(\tau) = \max_{v \in G_\tau} d(v), \quad (8)$$

where an empty maximum is zero and edges that represent only the serialized commit order are excluded. For a fork and join execution in which one orchestrator generation precedes two worker branches of length three and five and one integration generation follows, the critical path is $1 + \max(3, 5) + 1 = 7$ model rounds while serial execution of the same ten generations gives 10.

D REWARD COMPUTATION

Tool and Answer Matching. A reference milestone specifies a tool, its arguments, and the expected observation. Arguments are compared after declared defaults are filled and after the environment-specific normalization rules listed in the task package are applied, and observations are canonicalized and hashed. Only successful events contribute coverage, and each milestone is credited once, so repeated calls do not raise f_{tool} . The answer term is $f_{\text{ans}}(\tau) = \frac{m}{n^*} (0.8 + 0.2 \frac{m}{n})$, where n^* and n are the reference and submitted field counts and m is the matched count, so recall carries most of the weight and the precision factor removes any gain from padding. Scalar and structured values require equality, and list elements are matched one to one without reuse.

Delivery and Schedule. Each credited subtask has one assignment owner, and a repeated or empty instruction receives no ownership. Delivery checks whether the worker return contains the required raw evidence or a matching derived result, which is stricter than a claim of completion. For f_{sch} , a pair in Π counts toward Π_τ when the two subtasks are owned by different workers and the assignment intervals are causally independent, so issuing two assignments in sequence and waiting for the first does not count as parallel scheduling. Tasks with an empty Π fall back to a conservative schedule score based on covered milestones, satisfied dependencies, and non-redundant assignments.

Outcome Priority. Under Eq. 6, a successful team satisfies $f_{\text{out}} = 1$ and $f_{\text{proc}} \geq 0$, which gives $R \geq 1$, and an unsuccessful team satisfies $f_{\text{out}} = 0$ and $f_{\text{proc}} \leq 1$, which gives $R \leq \lambda$. Every verified success therefore outranks every failure, and the property assumes a correct outcome checker and bounded process terms.

E TRAINING AND EVALUATION CONFIGURATION

Every benchmark is evaluated with a frozen dataset revision, evaluator commit, subset manifest, and decoding configuration, and function-calling accuracy and task success are reported in separate columns with no average across heterogeneous metrics. The primary unit of analysis is the task, every method attempts the same instances under paired seeds, and uncertainty is estimated with paired bootstrap resampling of task identifiers. Infrastructure failures such as an unavailable service are logged separately from a valid trajectory that fails verification, and a rerun uses the same inputs and is counted in the run ledger. Benchmark tasks and benchmark-derived references are excluded from synthesis, from reward annotation, and from model selection.

Table 11: Training configuration of MAW. The orchestrator and the workers start from the same single-agent checkpoint, and only the orchestrator is updated.

Setting	Value
Backbone	Qwen3-8B
Orchestrator initialization	Single-agent checkpoint
Worker initialization and update	Same checkpoint, frozen
Optimization	GRPO with outcome and process reward
Rollouts per task	8
Process coefficient λ	0.3
Process weights $(\alpha_t, \alpha_a, \alpha_d, \alpha_s)$	(0.25, 0.25, 0.20, 0.30)
Cost in the training reward	None
Completion protocol	At least one delegation

F STATEMENTS

Reproducibility. The research package contains the environment and reset manifests, the task provenance and independence records, the synthesis and admission code, the reward implementation, the prompts, the baseline adapters, the training configurations, and the per-task evaluation logs. Hidden references are stored separately from the assets visible to a solver.

Ethics. Synthetic tool environments can encode unrealistic assumptions or faulty checks, so they are executed in isolated sandboxes and reviewed for copied private data, credential-like strings, and unauthorized external effects before release. The same orchestration capability that distributes useful work can distribute harmful actions, so deployment requires tool-level permissions and monitoring beyond the training reward.